Project: 1

Title: Java Application Deployment with Reverse Proxy on AWS

1. Working Application Link

The deployed application is accessible through the

reverse proxy server. Application URL:

http://<http://40.26.125.86/ /student

The reverse proxy forwards incoming requests to the backend application server running on Apache Tomcat.

2. Database Screenshot

The application stores submitted student registration data in the **Amazon RDS MySQL database**.

The following screenshot shows the records stored in the **students table**

after submitting the registration form. (Add Screenshot: rds-table.png)

Example query used to verify stored data:

```
1 row in set (0.002 sec)

MySQL [studentdb]> select * from students;
+----+--------+-----------------+-----------------+
| id | name   | email           | course          |
+----+--------+-----------------+-----------------+
|  1 | Aniket | aniket@test.com | Cloud Computing |
+----+--------+-----------------+-----------------+
1 row in set (0.002 sec)
```

SELECT * FROM students;

3. Explanation of Traffic Flow

The application uses a **reverse proxy architecture** to improve

security and manage traffic. Traffic flow:

User
↓
Nginx Reverse Proxy (EC2)
↓
Tomcat Application Server (EC2)
↓

Amazon RDS MySQL

When a user submits the form:

1. The request first reaches the **Nginx Reverse Proxy server**.
2. The proxy server forwards the request to the **backend Tomcat server** running on port **8080**.
3. The application processes the request and stores the data in **Amazon RDS MySQL database**.
4. The response is returned back to the user through the proxy server.

This architecture ensures that the **backend server is not directly exposed to the internet**.

4. Tools, Services, and Configurations Used

Cloud Services

- Amazon EC2 (Backend Server)
- Amazon EC2 (Reverse Proxy Server)
- Amazon RDS MySQL (Database)

Software & Tools

- Apache Tomcat
- Nginx Reverse Proxy
- Java JSP
- Linux (Amazon Linux)

Configurations

- Security groups configured to allow traffic only from proxy server to backend server
- Backend server running on **port 8080**
- Proxy server listening on **port**

5. Challenges Encountered and Solution

Challenge 1: Database connection issues

Initially the application was unable to store data in the database.

Solution:
Installed the MySQL connector for Java and configured the correct database endpoint.

Challenge 2: Reverse proxy configuration

Requests were not properly forwarded from the proxy server.

Solution:
Configured Nginx to route /student requests to the backend Tomcat server.

Challenge 3: Security group configuration

The backend server was initially accessible from the internet.

Solution:
Updated AWS security groups to allow access only from the proxy server.

6. Conclusion

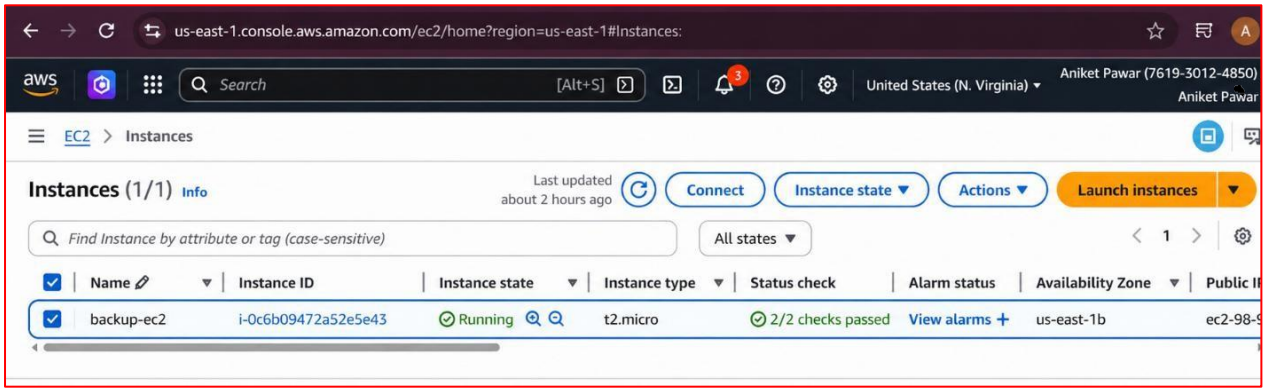
This project demonstrates the deployment of a **Java web application on AWS using a reverse proxy architecture**.

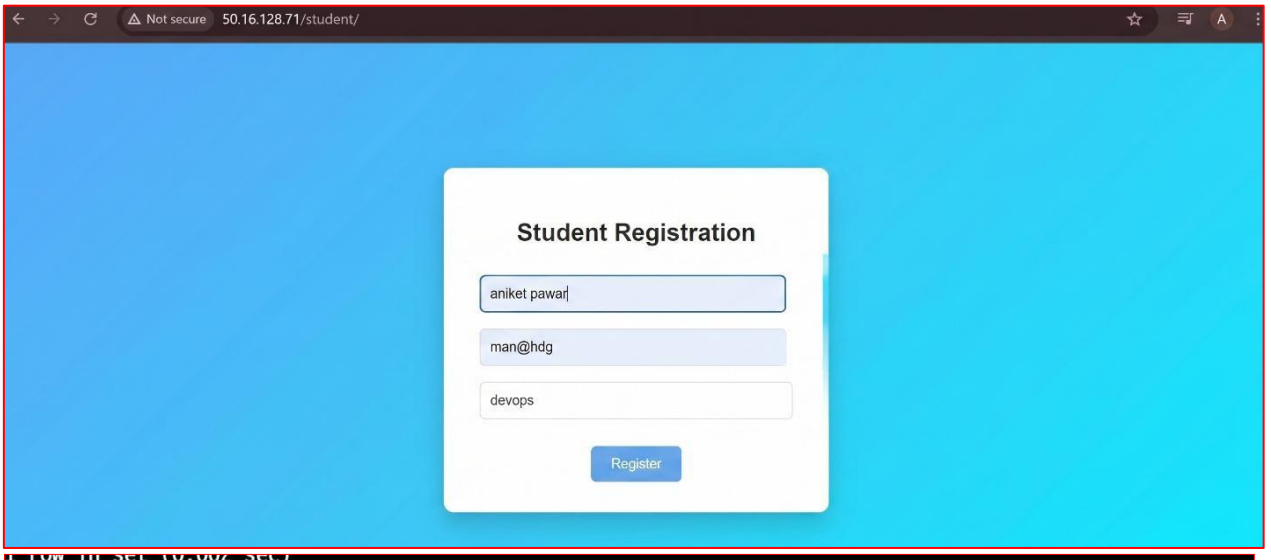
The implementation includes:

- Deploying the application on **Apache Tomcat**
- Configuring **Nginx reverse proxy**
- Connecting the application to **Amazon RDS MySQL**
- Implementing **secure architecture using security groups**

This project provided practical experience in **cloud deployment, reverse proxy configuration, and database integration**.

ScreenShots:






```
1 row in set (0.002 sec)

MySQL [studentdb]> select * from students;
+----+--------+----------------+----------------+
| id | name   | email          | course         |
+----+--------+----------------+----------------+
|  1 | Aniket | aniket@test.com | Cloud Computing |
+----+--------+----------------+----------------+
1 row in set (0.002 sec)
```

Project: 2

Title: Set Up AWS Backup Plan for EC2 and RDS

1. Objective

The objective of this project is to configure and manage automated backup and recovery for AWS resources using AWS Backup. The project demonstrates how to protect cloud infrastructure by creating a centralized backup plan for EC2 instances and RDS databases.

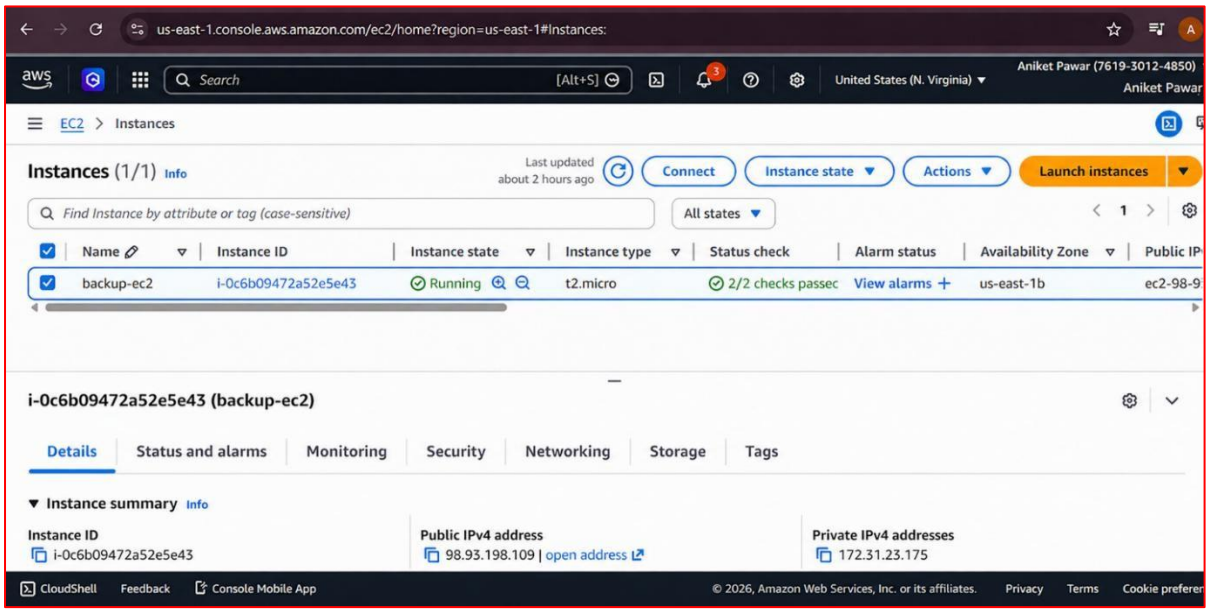
The goal is to ensure data protection, backup automation, and reliable recovery in case of data loss or system failure.

2. Infrastructure Setup

EC2 Instance

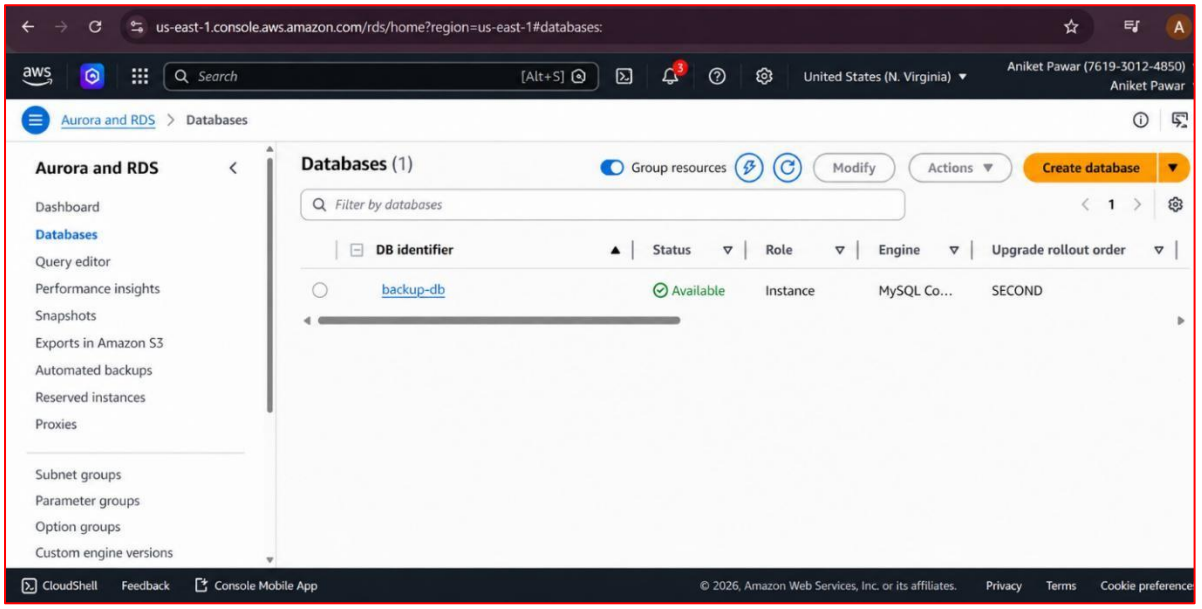
An Amazon EC2 instance was launched using Amazon Linux.
Apache Web Server was installed and a sample HTML page was created to simulate application data stored on the instance.

Screenshot:



RDS Database

An Amazon RDS MySQL database was created to store sample application data. A test database and table were created and sample records were inserted to verify database functionality.



AWS Backup Setup

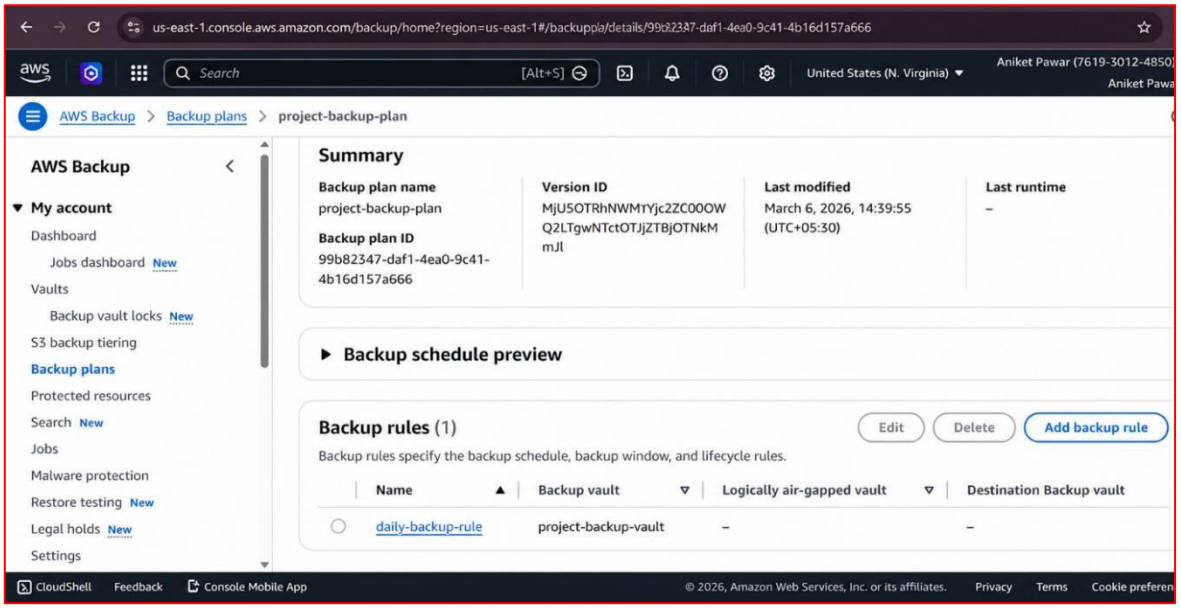
Backup Vault

A centralized Backup Vault named:

project-backup-vault

was created to securely store backup recovery points. The vault uses AWS-managed encryption keys to protect backup data.

Screenshot:



Backup Plan

A backup plan

named: project-

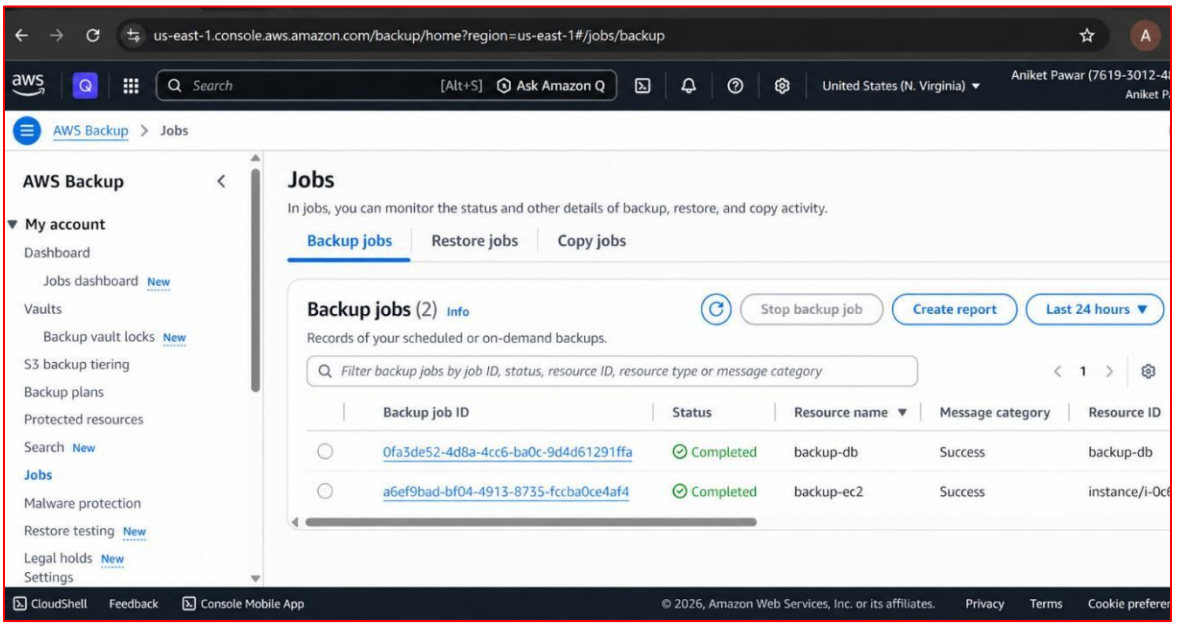
backup-plan

was created with the following configuration:

Backup Frequency: Daily
Backup Vault: project-backup-vault Retention Period: 7
Days

This plan automatically manages scheduled backups for selected resources.

Screenshot:



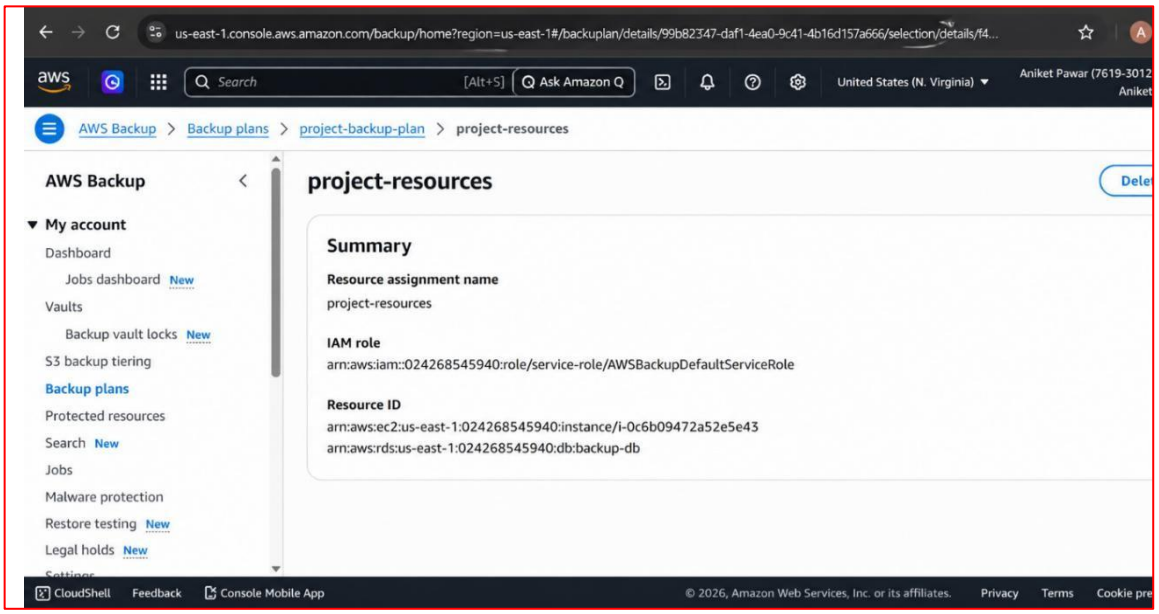
4. Resource Assignment

The following resources were assigned to the backup plan:

- EC2 Instance
- RDS Database (backup-db)

AWS Backup uses the AWSBackupDefaultServiceRole IAM role to create and manage recovery points for these resources.

Screenshot:

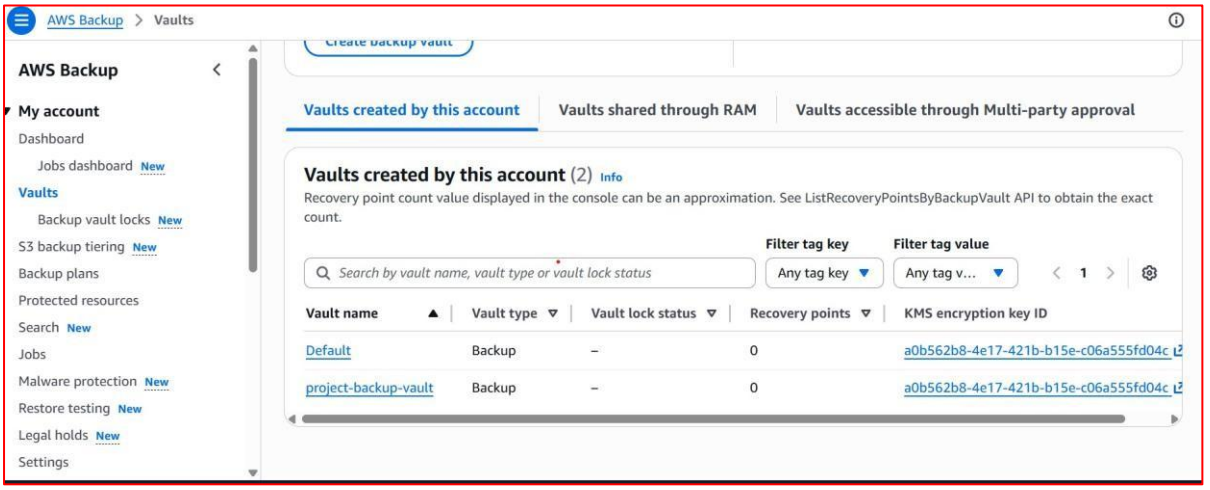


5. Backup Validation

To validate the configuration, on-demand backups were triggered manually.

The backup jobs were successfully executed and recovery points were created inside the backup vault.

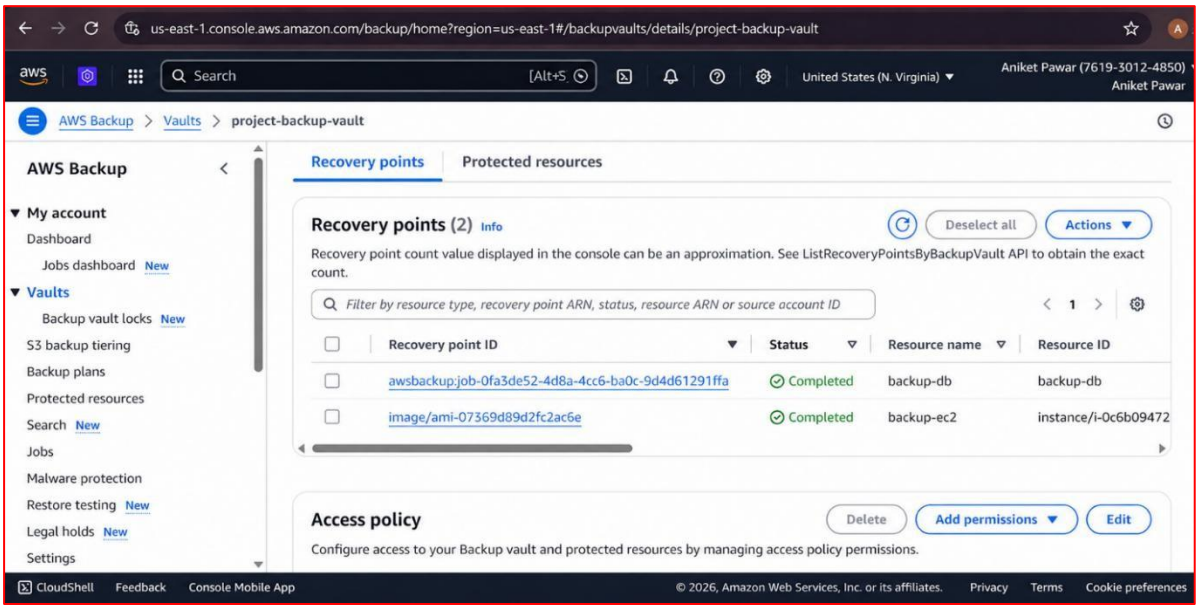
Screenshot:



6. Recovery Points Verification

The backup vault now contains recovery points for both EC2 and RDS resources, confirming that the automated backup configuration is working correctly.

Screenshots:



7. Tools and Services Used

- Amazon EC2
- Amazon RDS
- AWS Backup
- AWS IAM
- AWS Management Console

8. Conclusion

This project successfully demonstrated how to implement an automated backup strategy using AWS Backup. The EC2 instance and RDS database were protected using a centralized backup plan, and recovery points were verified to ensure data protection.

The implementation highlights the importance of automated backups in maintaining system reliability and enabling disaster recovery in cloud environments.

9. Author

Aniket Pawar
MCA Student | Cloud & DevOps Enthusiast

Project: 3

Title: Data Ingestion from S3 to RDS with Fallback to AWS Glue using Dockerized Python Application

1. Introduction

This project demonstrates a cloud-based data ingestion pipeline built using AWS services and a Dockerized Python application. The goal of the project is to read a CSV dataset from an Amazon S3 bucket and attempt to insert the data into an Amazon RDS MySQL database.

If the RDS database becomes unavailable or the insertion process fails, the application automatically falls back to AWS Glue Data Catalog. This ensures that the data pipeline remains operational even when the primary database is not accessible.

The project integrates multiple AWS services and demonstrates how to build a fault-tolerant data ingestion workflow using Python and Docker.

2. Objective

The main objectives of this project are:

- To read CSV data from an Amazon S3 bucket
- To process the data using Python and Pandas
- To insert the data into an Amazon RDS MySQL database
- To implement a fallback mechanism using AWS Glue when RDS fails
- To package and run the application using Docker

3. AWS Services Used

Amazon S3

Amazon S3 is used to store the CSV dataset that will be processed by the Python application.

Amazon RDS (MySQL)

Amazon RDS is used as the primary database where the CSV data is inserted.

AWS Glue Data Catalog

AWS Glue is used as a fallback mechanism to register the dataset if the RDS database becomes unavailable.

Amazon EC2

An EC2 instance is used to run the Docker container that executes the Python application.

AWS IAM

IAM is used to provide secure access to AWS services such as S3 and Glue

4. Technologies Used

- Python
- Pandas
- Boto3
- SQLAlchemy
- PyMySQL
- Docker
- AWS Cloud Services

5. System Architecture

The data pipeline follows the architecture below:

S3 Bucket → Dockerized Python Application → Amazon

RDS (MySQL) If RDS connection fails:

S3 Bucket → Dockerized Python Application → AWS Glue Data Catalog

This architecture ensures that the pipeline remains functional even if the database connection fails.

6. Project Implementation

Step 1 – Upload CSV File to Amazon S3

A CSV dataset is uploaded to an Amazon S3 bucket which acts as the source of the data.

Step 2 – Python Application Development

A Python script is created that performs the following tasks:

- Connects to Amazon S3 using Boto3
- Reads the CSV file using Pandas
- Attempts to insert the data into an Amazon RDS MySQL database

Step 3 – Implement Fallback Mechanism

If the connection to RDS fails, the application automatically switches to AWS Glue and creates a database in the Glue Data Catalog.

Step 4 – Docker Containerization

The Python application is packaged into a Docker container using a Dockerfile. The container installs all required dependencies and runs the Python script when the container starts.

Step 5 – Running the Application

The Docker container is executed on an EC2 instance using environment variables to provide AWS credentials and configuration parameters.

7. Project Output

The application successfully reads the CSV dataset from Amazon S3 and attempts to insert the records into the RDS database.

When the RDS connection fails, the application triggers the fallback mechanism and creates a database in AWS Glue Data Catalog.

Example container

output: CSV loaded

from S3
RDS failed switching to Glue
Glue database created

This confirms that the fallback mechanism works correctly.

8. Challenges Faced

AWS Credential Configuration

Initially, the Docker container could not access AWS services. This issue was resolved by passing AWS credentials using environment variables.

RDS Connection Timeout

The application encountered connectivity issues with the RDS database. The fallback mechanism was implemented to ensure that the pipeline continues to function.

IAM Permissions

Proper IAM permissions were configured to allow the application to access S3 and AWS Glue.

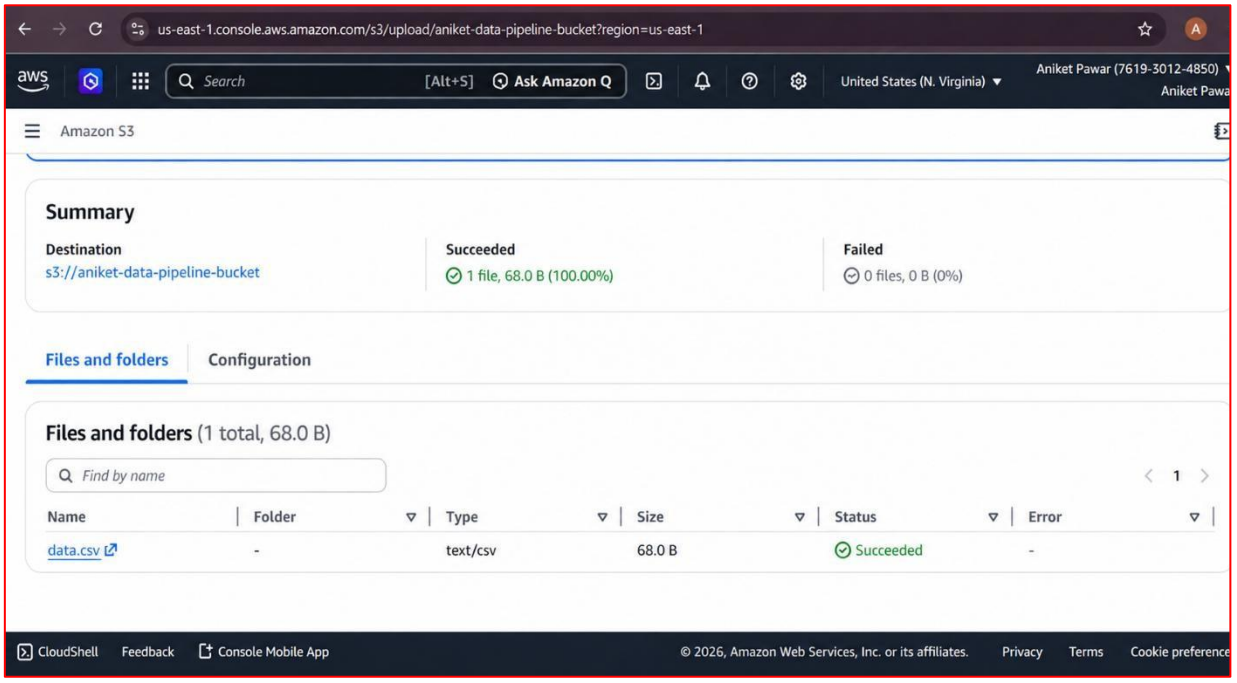
9. Conclusion

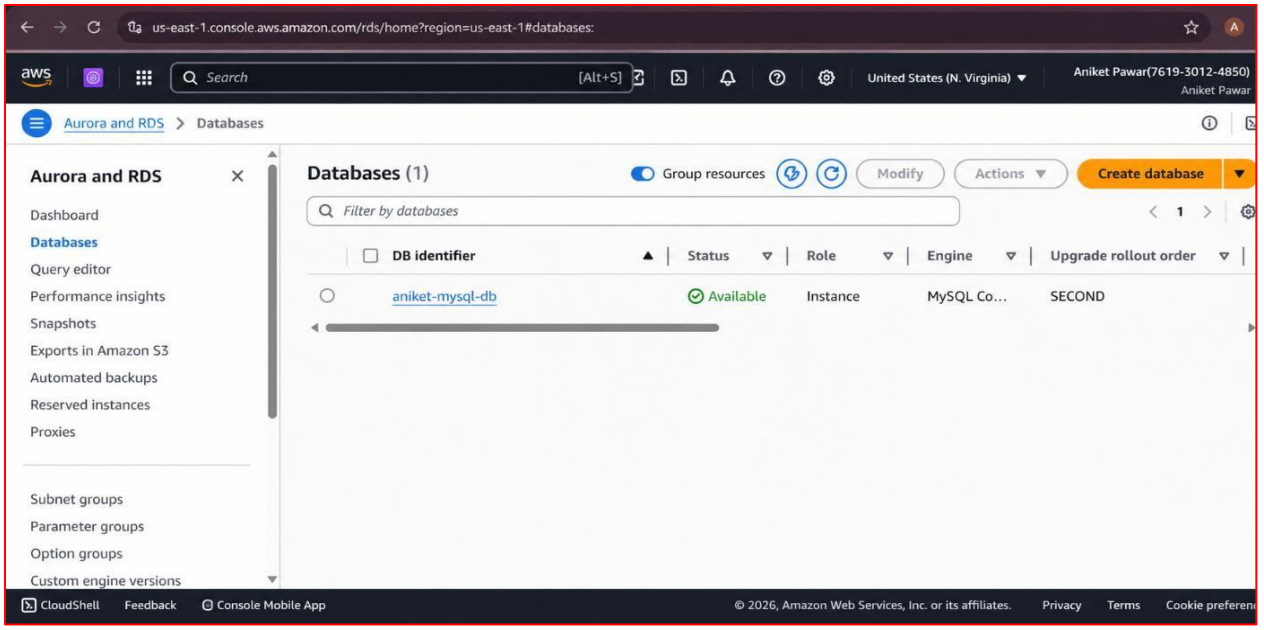
This project successfully demonstrates how to build a fault-tolerant cloud data ingestion pipeline using AWS services and Docker.

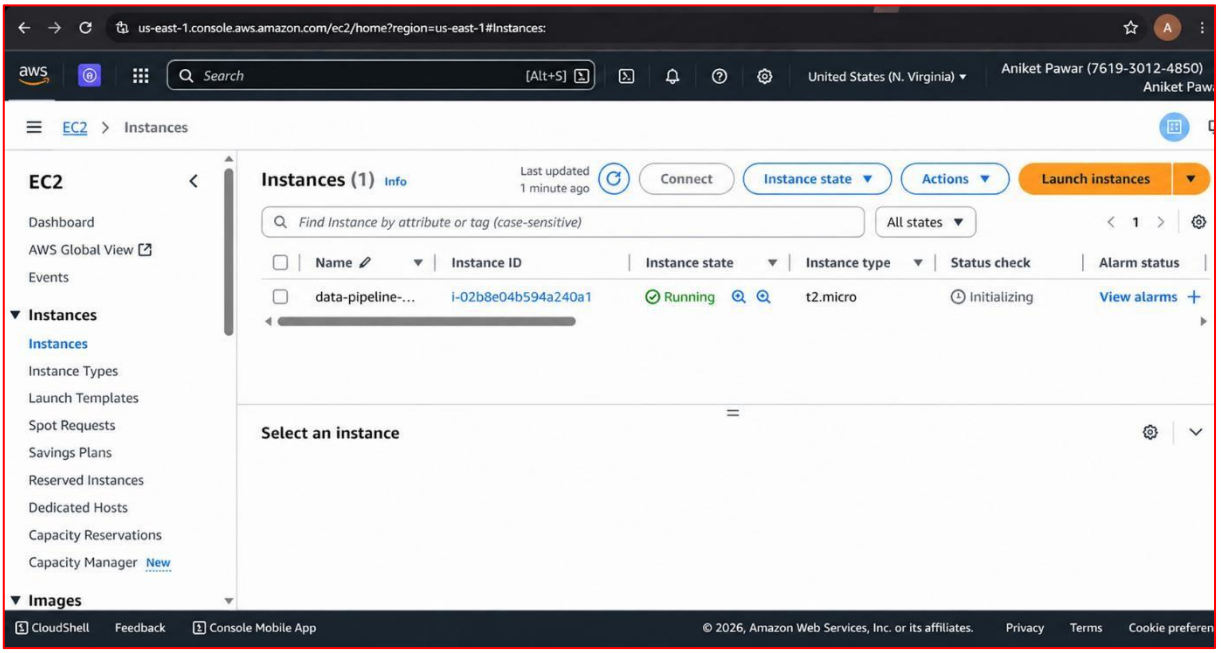
The application integrates Amazon S3, Amazon RDS, AWS Glue, and Docker to create a scalable and resilient data processing workflow. The fallback mechanism ensures that the system remains operational even when the primary database becomes unavailable.

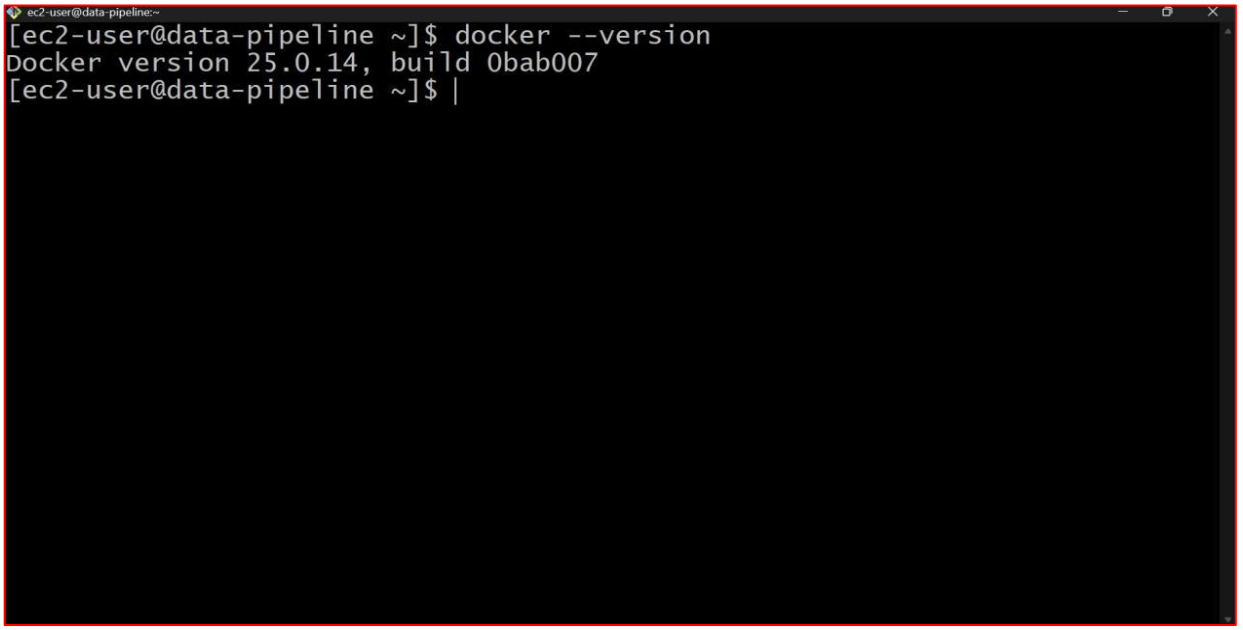
This project provides practical experience in working with AWS cloud services, building data pipelines, and containerizing applications using Docker.

Screenshots:






us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:
aws
Search
[Alt+S]
United States (N. Virginia)
Aniket Pawar (7619-3012-4850)
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
Instances (1) Info
Last updated 1 minute ago
Connect
Instance state
Actions
Launch instances
Find Instance by attribute or tag (case-sensitive)
All states
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
data-pipeline-...
i-02b8e04b594a240a1
Running
t2.micro
Initializing
View alarms
Select an instance
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferen

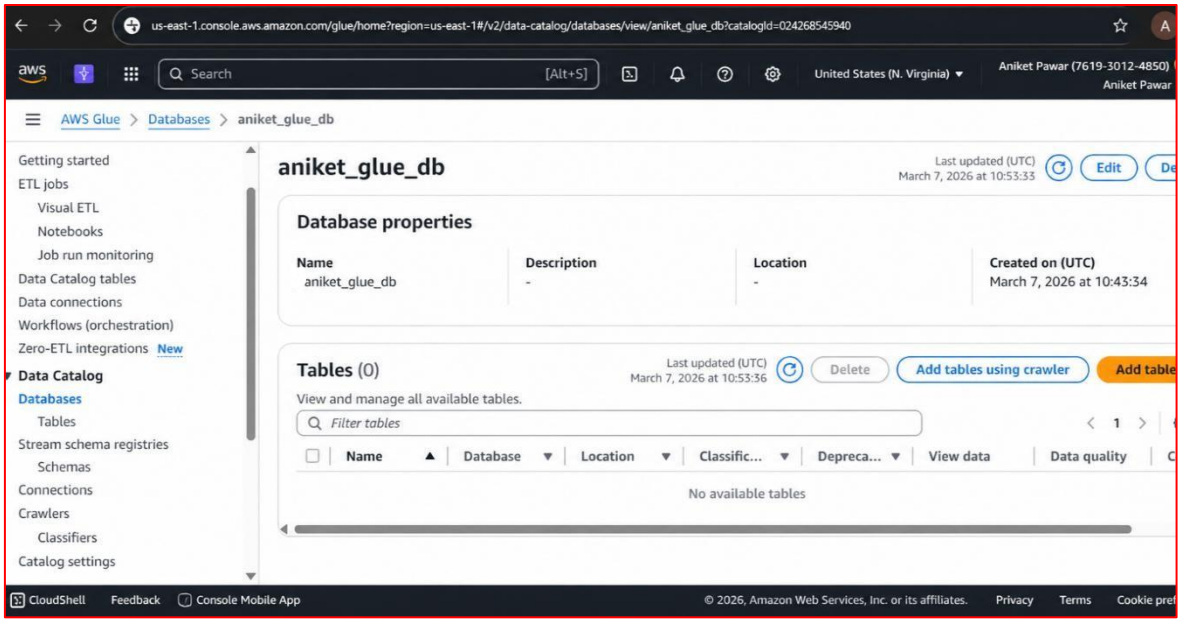

ec2-user@data-pipeline:~
[ec2-user@data-pipeline ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@data-pipeline ~]$

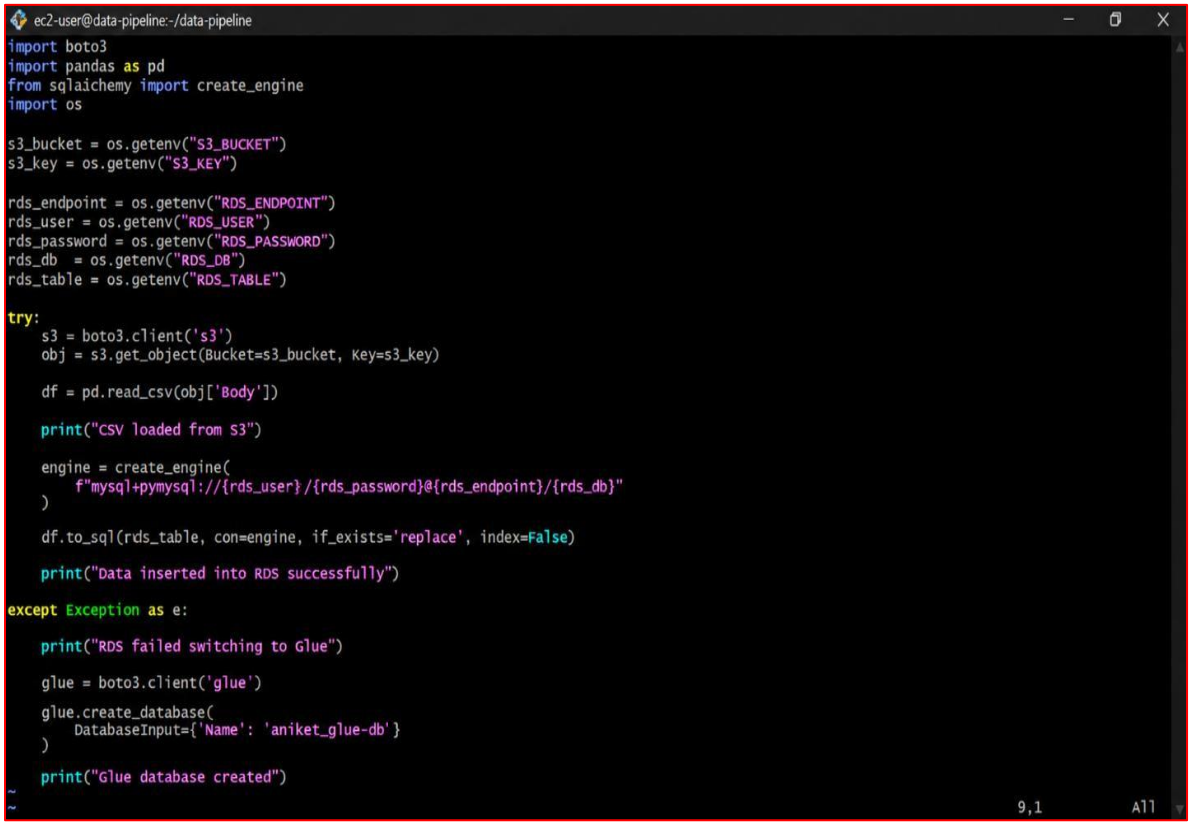
```
ec2-user@data-pipeline:~/data-pipeline
[ec2-user@data-pipeline ~]$ cd data-p
ipeline/
[ec2-user@data-pipeline data-pipeline
]$ ls
[ec2-user@data-pipeline data-pipeline]$ touch ap
p.py
[ec2-user@data-pipeline data-pipeline]$ touch Do
ckerfile
[ec2-user@data-pipeline data-pipeline]$ touch re
quirements.txt
[ec2-user@data-pipeline data-pipeline]$ ls
Dockerfile  app.py  requirements.txt
[ec2-user@data-pipeline data-pipeline]$ |
```

```
ec2-user@data-pipeline:~/data-pipeline
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python","app.py"]
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --                                    9,24          All
```

```
ec2-user@data-pipeline:~/data-pipeline
[ec2-user@data-pipeline data-pipeline]$ docker b
uild -t data-pipeline .
[+] Building 49.1s (9/9) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 198B                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.9                 0.4s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [1/4] FROM docker.io/library/python:3.9@sha256:da5aee29682d12a6649f51c   23.2s
 => => resolve docker.io/library/python:3.9@sha256:da5aee29682d12a6649f51c8   0.0s
 => => sha256:89d573bf42b377ce6a5a0451c15388849686fa4058e 25.62MB / 25.62MB   1.4s
 => => sha256:d6ca7d9522a172c424721d3509ee12079f7864a742b6a 2.32kB / 2.32kB   0.0s
 => => sha256:bcd3da5974912584a81ed86fd944ab5fba9093ff1c9a0 6.23kB / 6.23kB   0.0s
 => => sha256:26dfe2fac1c486e9aaf41d1028ed30be2c442aa84af 67.78MB / 67.78MB   1.7s
 => => sha256:da5aee29682d12a6649f51c8d6f15b87deb3e6c524b 10.30kB / 10.30kB   0.0s
 => => sha256:795dbedde24d2c72dafd2b71fe36643552e56859c0e 49.28MB / 49.28MB   1.3s
 => => sha256:79d5bd8a8d262418bf22e705535ce38c6789dc72e 235.93MB / 235.93MB   4.0s
 => => sha256:081ccf923272c30c6072c6ff1617d9072e03ab2a90a43 6.10MB / 6.10MB   1.7s
 => => extracting sha256:795dbedde24d2c72dafd2b71fe36643552e56859c0e29cdb09   4.1s
 => => sha256:c9723aa529b03c40e66d0aee927a410b4719528ab86 20.37MB / 20.37MB   2.1s
 => => sha256:91c91c91f1d23f4edf4280a8fe935f14340fec43a7a357614 250B / 250B   1.7s
 => => extracting sha256:89d573bf42b377ce6a5a0451c15388849686fa4058efd68999   1.0s
 => => extracting sha256:26dfe2fac1c486e9aaf41d1028ed30be2c442aa84af44462bc   3.5s
 => => extracting sha256:79d5bd8a8d262418bf22e705535ce38c6789dc72e319d76b3   10.7s
 => => extracting sha256:081ccf923272c30c6072c6ff1617d9072e03ab2a90a431951d   0.3s
 => => extracting sha256:c9723aa529b03c40e66d0aee927a410b4719528ab865af6e0b   1.1s
 => => extracting sha256:91c91c91f1d23f4edf4280a8fe935f14340fec43a7a3576149   0.0s
 => [internal] load build context                                             0.0s
 => => transferring context: 1.36kB                                           0.0s
 => [2/4] WORKDIR /app                                                        0.1s
 => [3/4] COPY . .                                                            0.1s
 => [4/4] RUN pip install -r requirements.txt                                20.2s
 => exporting to image                                                        5.0s
 => => exporting layers                                                       5.0s
 => => writing image sha256:327c00c30da12f61511f7b988f9df175d5165a272fc08c0   0.0s
 => => naming to docker.io/library/data-pipeline                              0.0s
[ec2-user@data-pipeline data-pipeline]$
```

```
CSV loaded from S3
RDS failed switching to Glue
Glue database created
[ec2-user@data-pipeline ~]$
```


us-east-1.console.aws.amazon.com/glue/home?region=us-east-1#/v2/data-catalog/databases/view/aniket_glue_db?catalogId=024268545940
Search
[Alt+S]
United States (N. Virginia)
Aniket Pawar (7619-3012-4850)
Aniket Pawar
AWS Glue > Databases > aniket_glue_db
Getting started
ETL jobs
Visual ETL
Notebooks
Job run monitoring
Data Catalog tables
Data connections
Workflows (orchestration)
Zero-ETL integrations New
Data Catalog
Databases
Tables
Stream schema registries
Schemas
Connections
Crawlers
Classifiers
Catalog settings
aniket_glue_db
Last updated (UTC)
March 7, 2026 at 10:53:33
Edit
Database properties
Name
aniket_glue_db
Description
-
Location
-
Created on (UTC)
March 7, 2026 at 10:43:34
Tables (0)
Last updated (UTC)
March 7, 2026 at 10:53:36
Delete
Add tables using crawler
View and manage all available tables.
Filter tables
Name
Database
Location
Classific...
Depreca...
View data
Data quality
No available tables
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms


ec2-user@data-pipeline:~/data-pipeline
import boto3
import pandas as pd
from sqlaichemy import create_engine
import os
s3_bucket = os.getenv("S3_BUCKET")
s3_key = os.getenv("S3_KEY")
rds_endpoint = os.getenv("RDS_ENDPOINT")
rds_user = os.getenv("RDS_USER")
rds_password = os.getenv("RDS_PASSWORD")
rds_db = os.getenv("RDS_DB")
rds_table = os.getenv("RDS_TABLE")
try:
    s3 = boto3.client('s3')
    obj = s3.get_object(Bucket=s3_bucket, Key=s3_key)
    df = pd.read_csv(obj['Body'])
    print("CSV loaded from S3")
    engine = create_engine(
        f"mysql+pymysql://{rds_user}/{rds_password}@{rds_endpoint}/{rds_db}"
)
    df.to_sql(rds_table, con=engine, if_exists='replace', index=False)
    print("Data inserted into RDS successfully")
except Exception as e:
    print("RDS failed switching to Glue")
    glue = boto3.client('glue')
    glue.create_database(
        DatabaseInput={'Name': 'aniket_glue-db'}
)
    print("Glue database created")
9,1
All